claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-a5773f24dc6dccc7c.jsonl`
- SHA-256: `8d2440b4e31f564e3900dd95926e4807bdf717bc384f11bb064dade8e3114b6c`
- Source modified: `2026-06-25T17:02:26+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T17:01:08.861Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with git diff origin/master...HEAD + Read the files). Be skeptical ? default to isReal=false / adjustedSeverity=INVALID if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [MEDIUM] Startup sweep can BLOCK boot ? files.list()/delete() have no timeout (the one real gap in the 'never blocks startup' claim)
Location: backend/providers/gemini.py:34,61,78 + backend/main.py:1345
Detail: _maybe_sweep_gemini_orphans runs SYNCHRONOUSLY at backend/main.py:1345, before uvicorn.run(app) at line 1461 ? so anything that stalls the sweep stalls the entire server/app boot. The sweep's network calls are unbounded: cleanup_orphaned_files calls list(self.client.files.list()) (gemini.py:61) and self.client.files.delete(...) (gemini.py:78) with NO timeout. The genai client is built at gemini.py:34 as genai.Client(api_key=self.api_key) with no http_options, and HttpOptions.timeout defaults to None (verified in google-genai 2.4.0), so a hung TCP connection or a slow multi-page files.list() pagination can block indefinitely. This directly contradicts the docstring's 'never blocks startup' (gemini.py:52, main.py:1190). Note the contrast: analyze_video deliberately wraps every Gemini network call in ExecutionPolicy/run_bounded with a HARD op_timeout_sec (gemini.py:132,166,178) precisely to be a 'hang-killer' ? the sweep skips that discipline entirely. 'Never raises' is satisfied (double try/except), but 'never hangs' is not ? and a hung boot is arguably worse than a crash (no terminal state, no loud failure). FIX: bound it. Construct/scope the client with a request timeout (genai.Client(..., http_options=types.HttpOptions(timeout=<ms>)) ? note units are MILLISECONDS) for the sweep, and/or wrap the whole sweep in run_bounded with a small op_timeout_sec (e.g. 10-15s) so a stalled File API can never gate startup. A wall-clock cap on the loop (delete-N-then-stop) also caps a pathological large-account sweep.
Suggested fix: Give the sweep a hard time bound: build the genai client with HttpOptions(timeout=...) (milliseconds) and/or run cleanup_orphaned_files under run_bounded(op_timeout_sec=~10-15s) so a slow/hung files.list()/delete() cannot block uvicorn boot.

Return isReal + adjustedSeverity (INVALID if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T17:01:11.385Z)

I'll verify this finding by examining the actual code in PR #380.

3. user (2026-06-25T17:01:13.132Z)

f00b71e feat(O1/P11): Gemini remote-file GC ? startup orphan sweep
o1-p11/gemini-remote-file-gc

4. user (2026-06-25T17:01:15.586Z)

```
 backend/main.py               |  25 ++++++++++
 backend/providers/gemini.py   |  43 ++++++++++++++++++
 tests/test_provider_gemini.py | 104 ++++++++++++++++++++++++++++++++++++++++++++
 3 files changed, 172 insertions(+)
```

5. assistant (2026-06-25T17:01:18.110Z)

Let me read the actual changed code in both files.

6. user (2026-06-25T17:01:19.179Z)

```
diff --git a/backend/main.py b/backend/main.py
index 5561cee..ea5a875 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
 from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
 from backend.reporting import emit_indexer_report
 from backend.results import Failure  # standardized error/result contract
 from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
         sys.exit(1)


+def _maybe_sweep_gemini_orphans(config) -> None:
+    """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+
 def main():
     parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
     if swept:
         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)
+
     # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
     if args.video_path and not args.server and not args.app:
         print(f"[CLI] Processing video file: {args.video_path}")
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
```

```
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
 import logging
 import os
 import time
+from datetime import datetime, timedelta, timezone
 from typing import Tuple

 from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
         except Exception:
             pass

+    def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+        """Sweep and delete video files orphaned on the Gemini File API by a previous
+        crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+        leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+        that reach a ``finally``/except).
+
+        Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+        creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+        Never raises; returns the count deleted. Intended to run once at startup.
+
+        Args:
+            grace_sec: only delete files older than this (default 1h). Protects a file a
+                live worker may still be processing.
+        """
+        cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+        deleted = 0
+        try:
+            files = list(self.client.files.list())
+        except Exception as e:
+            logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+            return 0
+        for f in files:
+            try:
+                mime = getattr(f, "mime_type", "") or ""
+                if not mime.startswith("video/"):
+                    continue
+                # f.create_time is an RFC3339 string on the File API.
+                ct = getattr(f, "create_time", None)
+                if ct is None:
+                    # No timestamp ? can't prove it's stale; leave it (safe default).
+                    continue
+                created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+                if created > cutoff:
+                    continue  # too recent ? a live worker may own it
+                self.client.files.delete(name=f.name)
+                deleted += 1
+            except Exception as e:
+                logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+        if deleted:
+            logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+        return deleted
+
     def analyze_video(self, video_path: str, prompt: str,
                       chunk_duration: float = 0.0,
                       label: str = "") -> Tuple[list, dict]:
```

7. assistant (2026-06-25T17:01:25.234Z)

The diff confirms the core mechanism. Now let me read the surrounding code in `gemini.py` to understand the client construction, the `analyze_video` ExecutionPolicy usage, and the

docstrings cited.

8. user (2026-06-25T17:01:25.934Z)

```
1       import json
2       import logging
3       import os
4       import time
5       from datetime import datetime, timedelta, timezone
6       from typing import Tuple
7
8       from google import genai
9       from google.genai import types
10
11      from backend.infrastructure.execution_policy import (
12          ExecutionPolicy,
13          PolicyTimeout,
14          poll_until,
15          run_bounded,
16      )
17      from backend.providers.base import AIVideoProvider
18
19      logger = logging.getLogger(__name__)
20
21      # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
22      # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
23      MAX_UPLOAD_MB = 500
24
25      class GeminiProvider(AIVideoProvider):
26          def __init__(self, api_key: str, model_name: str):
27              super().__init__(api_key, model_name)
28              # Initialize client lazily to avoid connection/auth issues during class
instantiation
29              self._client = None
30
31          @property
32          def client(self):
33              if self._client is None:
34                  self._client = genai.Client(api_key=self.api_key)
35              return self._client
36
37          def _safe_delete(self, video_file) -> None:
38              """Best-effort cleanup of an uploaded File (never raises)."""
39              try:
40                  self.client.files.delete(name=video_file.name)
41              except Exception:
42                  pass
43
44          def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
45              """Sweep and delete video files orphaned on the Gemini File API by a previous
46              crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
47              leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
48              that reach a ``finally``/except).
49
50              Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
whose
51              creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
upload).
52              Never raises; returns the count deleted. Intended to run once at startup.
53
54              Args:
55                  grace_sec: only delete files older than this (default 1h). Protects a file a
56                      live worker may still be processing.
57              """
```

```
            cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
            deleted = 0
            try:
                files = list(self.client.files.list())
            except Exception as e:
                logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
%s", e)
                return 0
            for f in files:
                try:
                    mime = getattr(f, "mime_type", "") or ""
                    if not mime.startswith("video/"):
                        continue
                    # f.create_time is an RFC3339 string on the File API.
                    ct = getattr(f, "create_time", None)
                    if ct is None:
                        # No timestamp ? can't prove it's stale; leave it (safe default).
                        continue
                    created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
                    if created > cutoff:
                        continue  # too recent ? a live worker may own it
                    self.client.files.delete(name=f.name)
                    deleted += 1
                except Exception as e:
                    logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
            if deleted:
                logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
            return deleted

        def analyze_video(self, video_path: str, prompt: str,
                          chunk_duration: float = 0.0,
                          label: str = "") -> Tuple[list, dict]:
            log_prefix = f"Gemini {label}" if label else "Gemini"

            metrics: dict = {
                "upload_time": 0.0,
                "process_time": 0.0,
                "gen_time": 0.0,
            }

            # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
            try:
                size_mb = os.path.getsize(video_path) / (1024 * 1024)
            except OSError:
                size_mb = 0.0
            if size_mb > MAX_UPLOAD_MB:
                logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
                             f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
                             f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
                metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
                metrics["oversize_skipped"] = True
                return [], metrics

            # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
            logger.info(f"[{log_prefix}] Uploading {video_path}...")
            t_upload_start = time.time()
            video_file = None
            for attempt in range(3):
                try:
                    video_file = self.client.files.upload(file=video_path)
```

```
                    metrics["upload_time"] = time.time() - t_upload_start
                    logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
                    break
                except Exception as e:
                    logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
                    if attempt < 2:
                        time.sleep(2 * (attempt + 1))
            if video_file is None:
                logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
                metrics["error"] = "upload failed after 3 attempts"
                return [], metrics

            # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
            # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
            t_process_start = time.time()
            process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0)  #
20 min cap

            def _processed():
                nonlocal video_file
                video_file = self.client.files.get(name=video_file.name)
                return None if video_file.state.name == "PROCESSING" else video_file

            try:
                video_file = poll_until(_processed, process_policy,
                                        label=f"{log_prefix} processing", logger=logger)
            except PolicyTimeout:
                logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
                self._safe_delete(video_file)
                metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
                return [], metrics
            except Exception as e:
                logger.error(f"[{log_prefix}] Status check failed: {e}")
                self._safe_delete(video_file)
                metrics["error"] = f"status check failed: {e}"
                return [], metrics
            if video_file.state.name == "FAILED":
                logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
                self._safe_delete(video_file)
                metrics["error"] = f"video processing failed: {video_file.error.message}"
                return [], metrics

            metrics["process_time"] = time.time() - t_process_start
            logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")

            # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
            # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
            # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
            logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
            t_gen_start = time.time()
            gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
            raw_text = None
```

```
168
169             def _generate():
170                 return self.client.models.generate_content(
171                     model=self.model_name,
172                     contents=[video_file, prompt],
173                     config=types.GenerateContentConfig(
174                         response_mime_type="application/json", temperature=0.0),
175                 )
176
177             try:
178                 response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
generate", logger=logger)
179                 raw_text = response.text
180                 metrics["gen_time"] = time.time() - t_gen_start
181                 logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
182             except Exception as e:                  # PolicyTimeout (hang) or the last 503 ?
both end here
183                 logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
184                 metrics["error"] = f"generate failed after retries: {e}"
185                 metrics["generate_failed"] = True
186             self._safe_delete(video_file)
187
188             if not raw_text:
189                 return [], metrics
190
191             # 4. Parse JSON
192             try:
193                 segments = json.loads(raw_text)
194                 if not isinstance(segments, list):
195                     logger.error(f"[{log_prefix}] Response is not a JSON array.")
196                     metrics["error"] = "response is not a JSON array"
197                     return [], metrics
198                 return segments, metrics
199             except Exception as e:
200                 logger.error(f"[{log_prefix}] JSON parse failed: {e}")
```

9. assistant (2026-06-25T17:01:29.043Z)

Now I've confirmed the key facts. Let me verify the docstring claims about "never blocks startup" cited at gemini.py:52 and main.py:1190, and look at the main.py boot path to confirm the synchronous call happens before uvicorn.run.

10. user (2026-06-25T17:01:30.457Z)

```
1185             sys.exit(1)
1186
1187
1188     def _maybe_sweep_gemini_orphans(config) -> None:
1189         """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
crashed run.
1190         A no-op unless the configured provider is Gemini AND its API key is present. Never
raises ?
1191         a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
tests."""
1192         try:
1193             provider_name = getattr(config, "ai_provider", "")
1194             if provider_name != "gemini":
1195                 return
1196             api_key = os.environ.get("GEMINI_API_KEY")
1197             if not api_key:
1198                 return
1199             provider = provider_registry.get("gemini", api_key=api_key,
model_name=config.ai_model)
1200             deleted = provider.cleanup_orphaned_files()
```

```
1201            if deleted:
1202                logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).",
deleted)
1203        except Exception as e:
1204            logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1205
1206
1207    def main():
1208        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
1209        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
```

11. user (2026-06-25T17:01:31.442Z)

```
1461:        uvicorn.run(app, host=bind_host, port=port)
```

12. assistant (2026-06-25T17:01:35.014Z)

Now let me look at the exact docstring wording. The finding claims gemini.py:52 and main.py:1190 say "never blocks startup." Let me check precisely what is claimed.

The docstring at gemini.py:52 says "Never raises; returns the count deleted. Intended to run once at startup." ? it does NOT say "never blocks startup." The main.py:1190 docstring says "Never raises ? a failed sweep is logged and swallowed so it can't block startup." And the inline comment at main.py:1344 says "never blocks startup."

Let me verify the exact wording of the inline comment and confirm the call ordering relative to uvicorn.run.

13. user (2026-06-25T17:01:36.439Z)

```
1340            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
1341
1342        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1343        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1344        # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1345        _maybe_sweep_gemini_orphans(global_config)
1346
1347        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1348        if args.video_path and not args.server and not args.app:
1349            print(f"[CLI] Processing video file: {args.video_path}")
```

14. user (2026-06-25T17:01:36.902Z)

```
1450                    print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1451                          f"gate (edge_auth_enabled / a reverse proxy) is in front, and set
"
1452                          f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header
guard defaults to "
1453                          f"loopback-only and will 400 forwarded requests otherwise).")
1454            # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1455            # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1456            # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1457            _get_executor().submit(_drain_due_jobs, global_config, db_instance)
1458            # Stamp shared state on the app so route handlers access it via
request.app.state
```

```
1459            # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
1460            create_app(global_config, db_instance)
1461            uvicorn.run(app, host=bind_host, port=port)
1462
1463    if __name__ == "__main__":
1464        # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
```

15. assistant (2026-06-25T17:01:41.007Z)

Confirmed. The call ordering and claims are exactly as the finding describes. Now let me verify the key technical claim: that `genai.Client(api_key=...)` is built with no `http_options`, and whether `files.list()`/`files.delete()` have any timeout. Let me check the installed google-genai version and how the client/files handle timeouts.

16. user (2026-06-25T17:01:44.831Z)

```
genai version: 2.4.0
```

17. user (2026-06-25T17:01:46.973Z)

```
HttpOptions.timeout default: None
HttpOptions.timeout annotation: typing.Optional[int]
```

18. assistant (2026-06-25T17:01:51.322Z)

Confirmed: google-genai 2.4.0 is installed, `HttpOptions.timeout` defaults to `None` (Optional[int], in milliseconds per the SDK). Let me verify the client is built without http_options (so the default None timeout applies) and that `run_bounded` is the discipline `analyze_video` uses but the sweep doesn't. I've already seen the client construction at line 34 (`genai.Client(api_key=self.api_key)` ? no http_options). Let me confirm `run_bounded` semantics briefly to validate the proposed fix is sound.

19. user (2026-06-25T17:01:52.552Z)

```
diff --git a/tests/test_provider_gemini.py b/tests/test_provider_gemini.py
index 141af81..5df2ef4 100644
--- a/tests/test_provider_gemini.py
+++ b/tests/test_provider_gemini.py
@@ -101,3 +101,107 @@ def test_gemini_provider_upload_fail():
         provider = GeminiProvider(api_key="dummy_api_key", model_name="gemini-2.5-flash")
         segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
         assert len(segments) == 0
+
+
+
+# --- O1/P11: Gemini remote-file GC (startup orphan sweep) ---
+
+def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
+    f = MagicMock()
+    f.name = name
+    f.mime_type = mime
+    f.create_time = create_time_iso
+    f.state = MagicMock()
+    f.state.name = state
+    return f
+
+
+def test_cleanup_orphaned_files_deletes_stale_videos_only():
+    """Deletes video files older than the grace window; leaves non-videos and fresh files."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
+    files = [
```

```
+        _file_mock("old-vid", "video/mp4", stale),       # stale video -> deleted
+        _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
+        _file_mock("old-img", "image/png", stale),        # non-video -> kept
+        _file_mock("notts", "video/mp4", None),           # no timestamp -> kept (safe)
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1
+    provider._client.files.delete.assert_called_once_with(name="old-vid")
+
+
+def test_cleanup_orphaned_files_list_failure_is_swallowed():
+    """A list() failure must not raise ? the sweep is best-effort and must not block
startup."""
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.side_effect = RuntimeError("network")
+        assert provider.cleanup_orphaned_files() == 0
+
+
+def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
+    """A delete() failure on one file must not abort the sweep."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    files = [
+        _file_mock("a", "video/mp4", stale),
+        _file_mock("b", "video/mp4", stale),
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        # First delete raises, second succeeds ? _safe_delete swallows it.
+        provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1  # only the second file counted
+
+
+
+# --- O1/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
+
+def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
+    """A non-Gemini provider must short-circuit (no client built, no sweep)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "openai"
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []  # provider never instantiated
+
+
+def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
+    """No GEMINI_API_KEY -> no sweep (can't auth)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    monkeypatch.delenv("GEMINI_API_KEY", raising=False)
+    called = []
```

```
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []
+
+
+def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
+    """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
startup)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    cfg.ai_model = "gemini-2.5-flash"
+    provider = MagicMock()
+    provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
+    monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    # Must not raise.
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    provider.cleanup_orphaned_files.assert_called_once()
```

20. assistant (2026-06-25T17:02:06.669Z)

The tests confirm the behavior: they only test that exceptions are swallowed (mocked side effects), but there is no test for a *hang* ? because a hang can't be unit-tested easily. This corroborates the finding's claim that "never raises" is covered but "never hangs" is not.

I now have everything needed to render a verdict. Let me consolidate the verification:

Verified facts:
1. `_maybe_sweep_gemini_orphans` is called synchronously at `backend/main.py:1345`, before `uvicorn.run(app, ...)` at line 1461 ? confirmed. A blocking sweep blocks boot.
2. `cleanup_orphaned_files` calls `list(self.client.files.list())` (line 61) and `self.client.files.delete(name=f.name)` (line 78) ? confirmed, no per-call timeout.
3. The client is built at `gemini.py:34` as `genai.Client(api_key=self.api_key)` with no `http_options` ? confirmed.
4. `HttpOptions.timeout` defaults to `None` in the installed google-genai 2.4.0 ? confirmed by runtime inspection.
5. The try/except in `cleanup_orphaned_files` and the outer one in `_maybe_sweep_gemini_orphans` guarantee "never raises" but do nothing against an indefinite hang (a stalled socket or slow pagination blocks inside the `try`).
6. The inline comment at `main.py:1344` literally claims "never blocks startup," and the docstring at `main.py:1190` says "it can't block startup" ? directly contradicted by the unbounded network calls.
7. `analyze_video` does use `run_bounded`/`ExecutionPolicy` with hard timeouts (lines 132, 166, 178) ? the discipline the sweep skips, confirming the contrast the finding draws.

Minor nit on the finding's precision: It cites gemini.py:52 as a "never blocks startup" docstring; that exact line actually says "Intended to run once at startup" (not the "never blocks" claim). The "never blocks startup" wording lives at main.py:1190 and the comment at main.py:1344. This is a small attribution imprecision but does not undermine the substance ? the contradiction is real and present in the code.

The finding is technically accurate and the risk is real. Severity assessment: this is a best-effort, default-OFF-unless-Gemini-configured maintenance sweep. It only triggers when provider is Gemini AND a key is present. A hang requires a pathological File API condition (hung TCP, very slow pagination). The consequence is a blocked boot ? genuinely bad (silent, no terminal state). MEDIUM is appropriate ? real but conditional and not a data-loss/security issue.

21. user (2026-06-25T17:02:15.011Z)

Structured output provided successfully

22. assistant (2026-06-25T17:02:26.251Z)

Verified the finding against the real PR #380 code. It is REAL at MEDIUM severity.

Key confirmations:
- `backend/main.py:1345` calls `_maybe_sweep_gemini_orphans(global_config)` synchronously, before `uvicorn.run(app, ...)` at `backend/main.py:1461` ? a stalled sweep blocks boot.
- `backend/providers/gemini.py:61` (`list(self.client.files.list())`) and `:78` (`self.client.files.delete(name=f.name)`) have no per-call timeout.
- `backend/providers/gemini.py:34` builds the client as `genai.Client(api_key=self.api_key)` with no `http_options`; I confirmed at runtime that installed google-genai 2.4.0 has `HttpOptions.timeout` defaulting to `None` ? so a hung socket or slow pagination can block indefinitely.
- The double try/except gives "never raises" but not "never hangs" (the stall is inside the `try`), directly contradicting the "never blocks startup" claims at `main.py:1344` (comment) and `main.py:1190` (docstring).
- `analyze_video` does wrap its Gemini calls in `run_bounded`/`ExecutionPolicy` hard timeouts (gemini.py:132,166,178) ? discipline the sweep omits.

One minor attribution slip in the finding: it cites `gemini.py:52` as a "never blocks startup" docstring, but that line actually reads "Intended to run once at startup"; the "never blocks" wording is at `main.py:1190/1344`. Substance is unaffected. The suggested fix (HttpOptions timeout in ms and/or `run_bounded` with a ~10?15s `op_timeout_sec`) is sound.